TECHNICAL REPORT · VERSION 2.0 · 18 OCTOBER 2026

Personal AI Dashboard with Evidence-Grounded Analytics

Architecture, AI-assisted engineering with Claude Code, verification evidence and lessons learned

Author	Dor Meir
Built with	Claude Code: Architect → Coder → independent Verifier agents under an Integration Lead, with explicit owner approval gates
Source code	github.com/DorGM/personal-ai-dashboard-submission
Demo video	Personal-AI-Dashboard-Demo.mp4 (release `submission-v1`; Demo Mode without an OpenAI key)
Development records	DEVELOPMENT_LOG.md · PROJECT_PLAN.md · evidence index · curated prompts
Evidence basis	Application release `32d849a` of the private development repository, published as a single-commit public snapshot; all figures come from the recorded Verifier and test results

Evidence labels used throughout. TESTED a command, test suite or browser flow was executed and its result recorded · REVIEWED checked by reading, no automated run · PENDING designed or approved, not yet executed · NOT RUN deliberately not exercised.
Commit references. SHAs such as `32d849a` identify commits in the private development repository, whose history is preserved unchanged. The public repository contains the released code as a fresh snapshot plus documentation commits; the sanitized development log and plan cite the private SHAs.

Contents

1. Executive summary 2
2. Problem and objectives 2
3. Architecture 4
    3.1 Analytics architecture 4
4. Claude Code methodology 5
5. Prompts and iterations 6
6. Verification and testing 7
    6.3 Scripted-model tests and the live evaluation plan 8
7. Defects found by independent verification 8
8. Results and reproducibility 9
9. Critical reflection and risks 10
10. Conclusion 10
Appendix A. Decision register 11
Appendix B. Claude Code prompts, iterations and engineering decisions 12
Appendix C. Additional evidence 17

1. Executive summary

The Personal AI Dashboard is a local, private web application for analyzing one's own life: food and health, content consumption, job applications and tasks. It keeps deterministic history, statistics and charts, and adds an **Analyst** that answers natural-language questions only with what the records support. The runtime OpenAI agent may call six read-only, user-scoped tools; it never writes SQL and never writes numbers. The backend computes and renders every value, and a deterministic **Evidence Validation** layer (rules V1–V14) checks every claim before display. When the data cannot support an answer, the product shows a fixed, owner-specified message. A **Demo Mode** with a 30-day fictional dataset lets any reviewer run the product with no Google account and no API key.

The project was also an exercise in disciplined AI-assisted engineering: every design decision was proposed by a Claude Code Architect agent and approved by the owner, and every implementation task was closed only after an *independent* Verifier agent passed it. Verifiers rejected work more than ten times; each rejection exposed a defect that the implementing agent's own green test suite had missed (§7).

~1,740	**30+**	**0**	**2**
automated tests, full suite green; ruff and mypy --strict clean	defects caught by independent verification before commit	mismatches in 172,960 randomized and fixture metric checks	viewports verified in a real browser (1440×900, 390×844)

Objective	Outcome
Reliable personal record keeping	TESTED 11 record domains; manual create, edit, soft delete, restore and separately confirmed permanent delete; database-enforced uniqueness; data-preservation guards that survived six adversarial verification rounds.
Deterministic metrics and charts	TESTED Closed metric registry checked against an independent calculator; responsive dashboard with gaps shown as gaps, never zeros.
Grounded AI analytics	TESTED with a scripted model: tools, evidence ledger, validator, repair-then-degrade, user and demo isolation. NOT RUN live OpenAI inference and the offline golden question set (see the box below).
Reproducible reviewer experience	TESTED Demo Mode: checksum-locked fixture, fresh working copy per start, local-only isolation; fresh-clone smoke test of the public snapshot.

AI validation status. No live OpenAI call has been made in this project, and the offline golden set of about 20 questions was designed but not executed. Every Analyst result in this report comes from a *scripted* model that returns predefined tool calls and draft answers, driving the real tools, metrics, validator and UI. This proves the deterministic guarantees around the model; it does not measure how well a real model chooses tools or follows the placeholder protocol. Live evaluation was deliberately gated behind the owner's approval of paid API use (decision O-9). §6.3 describes the method that would be used.

Demo video. A 4 min 42 s walkthrough recorded by the owner on the release build in Demo Mode is published as a release asset. It shows the dashboard tabs, history and aggregates on synthetic data, and the Analyst's no-key message. It does not show live AI analysis.

2. Problem and objectives

People who log their sleep, meals, workouts, reading or job search usually end up with scattered records and no trustworthy way to ask "what actually changed?". General chat assistants answer fluently, but they invent numbers, ignore gaps in the data and imply causation. The product goal is the opposite: **a personal analyst that may only say what the records support**.

Intended experience

- **Record** daily activities through accessible forms on desktop or phone; duplicates are rejected with a clear message.
- **Review** a 15-day history per domain where an undocumented day shows a placeholder (in Hebrew, per the original specification) and is never a zero.
- **See trends** in aggregates whose denominators are documented days only.
- **Ask** a question and get a validated answer with coverage and evidence, a partial answer naming what is missing, or an honest "not enough data".

Domain rules that shape the AI design

Rule	Consequence for AI
Missing ≠ zero; denominators are documented days	The agent cannot turn "forgot to log" into "did nothing".
Database uniqueness is authoritative; no automatic overwrite	No model output can replace a stored record.
Retrieval and arithmetic are fixed, parameterized SQL	No model-generated SQL; user scoping cannot be bypassed by prompt text.

Scope evolution

The first approved scope (Phase A, 16 Oct) also contained *external* intelligence: job-recommendation and industry-trend agents using web search, and AI free-text intake. Two recorded owner decisions narrowed it. ADR-005 (17–18 Oct) refocused the product on analyzing the user's own data, described by the owner as a deliberate conceptual shift; LOG-019 then deferred AI free-text intake so that runtime AI is concentrated on analytics. Superseded text was kept and marked, never deleted.



Each step recorded verbatim; superseded text kept and marked, never deleted.

Figure 1. Requirements refinement timeline.

In the MVP	Deferred (history and tables kept)
Manual records for 11 domains with soft delete, restore and confirmed permanent delete · 15-day history · deterministic aggregates and SVG charts · Analyst (one function-calling agent, six read-only tools, Evidence Validation) · Demo Mode · Google sign-in for personal mode · responsive desktop and mobile UI	AI free-text intake and calorie estimation (B-06) · job recommendations and industry trends (B-08, B-09) · scheduler · multi-turn analyst conversations · optional Cloudflare tunnel check (B-11) · *rejected by design:* embeddings, vector DB, multi-agent frameworks, LLM-generated SQL

3. Architecture

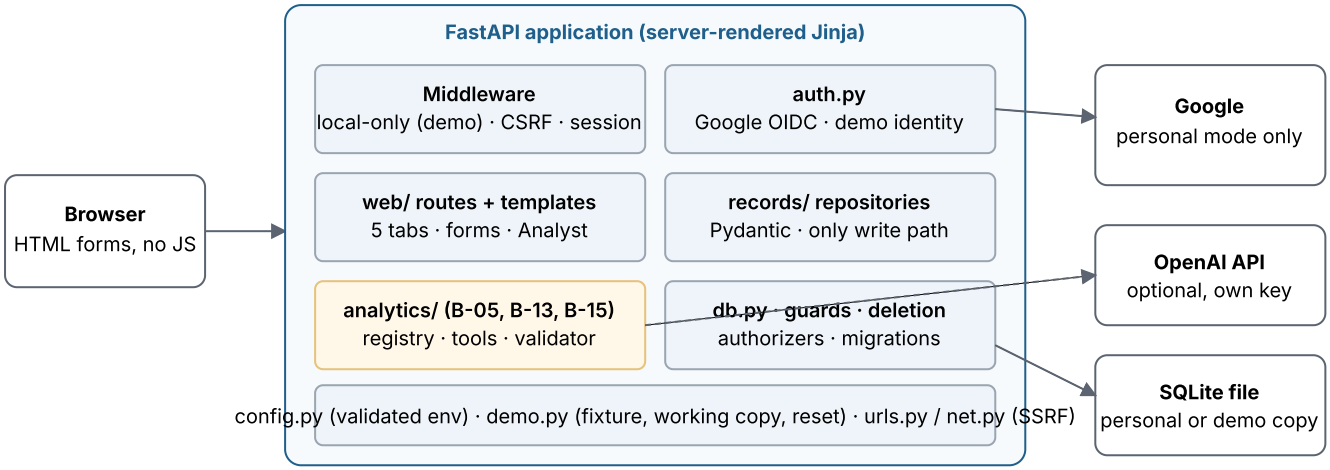


Figure 2. Component view. Amber = analytics package.

Concern	Decision and rationale	Record
Stack	Python FastAPI + Jinja/HTMX + SQLite, chosen over TypeScript/React and Next.js/Prisma: smallest dependency budget, no JavaScript build step, one process to start and stop.	A-02, LOG-003
Data model	17 STRICT tables and a `daily_calories` view; user-scoped canonical-key UNIQUE constraints; composite foreign keys; append-only calorie estimates (an owner correction, Appendix B #2).	A-03, B-03
Data preservation	RESTRICT foreign keys; deletes only through a grant-guarded `app/deletion.py`; REPLACE and UPSERT neutralized by triggers and a SQLite authorizer; immutable identities; migration invariants checked before COMMIT; app connections cannot run DDL.	ADR-004, F-01
Soft delete	Migration 002 adds `deleted_at`; Recently deleted view, explicit restore, separately confirmed permanent delete.	O-1, B-04
Authentication	Google OIDC (state, nonce, PKCE); allowlist bound to the stable `google_sub`; signed 8-hour cookie; CSRF middleware; test login only from a loopback peer.	B-02
Demo Mode	`APP_MODE=demo`: localhost only; refuses tunnel URLs and test login; demo marker in SQLite `application_id` enforces mode exclusivity; per-mode cookie and signing key with a `generation` claim; fresh working copy from a SHA-256-verified JSON fixture on every start.	ADR-005 §6, B-14
Deployment	Local process started and stopped by scripts; optional Cloudflare quick tunnel in personal mode only. No permanently hosted server.	A-04, B-01

3.1 Analytics architecture

ADR-005 separates **what the model decides** (which tools to call, how to phrase findings) from **what the backend guarantees** (every number, coverage statement, direction and caveat).

Draft answer from the model: text with placeholders such as {{E3.b.mean}} and cited evidence ids — never digits.

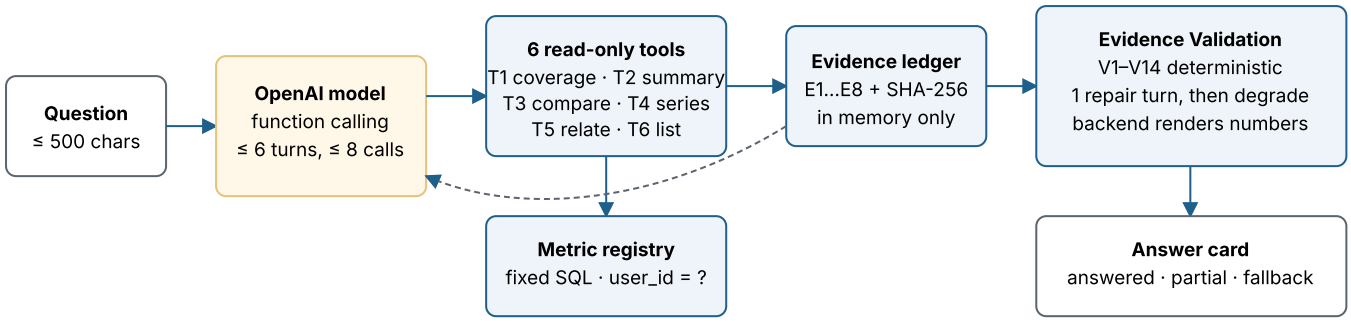


Figure 3. Request flow of the Analyst (ADR-005 §3.1). The dashed arrow returns tool results to the model as data envelopes.

- **Closed metric registry** shared with the dashboard (e.g. `sleep.hours`, `steps.count`, `tasks.completion_rate`), each with kind, unit and a minimum meaningful change.
- **Six tools** with strict argument models (`extra="forbid"`, so the model cannot pass a `user_id`); the user comes only from the server-side session. Queries are module-level constants chosen by enum, values bound with `?`; a static test bans string-built SQL.
- **Hardened read-only connection**: `query_only`, a deny-by-default authorizer, a 2-second statement budget, row caps, 8 KB result trimming. In Demo Mode the Analyst opens only the active demo working copy and fails closed otherwise.
- **Evidence ledger and placeholders.** Each tool result becomes an evidence item (`E1` … `E8`) with a SHA-256. The model writes "Average sleep was {{E3.b.mean}}" and cites `E3`; the backend substitutes formatted values and appends coverage lines and an "association only" caveat.

Rules	Requirement	Deterministic check (default thresholds)
V1–V2	Well-formed and supported	Strict schema; every finding cites existing evidence; every placeholder resolves to an allowed field of cited evidence.
V3	No free numbers	No digit or English number word outside placeholders.
V4–V5	Enough data	Value ≥ 3 observations; trend ≥ 7 points over ≥ 14 days at ≥ 50% coverage; correlation ≥ 10 pairs; groups ≥ 5 days.
V6–V7	Missing days; comparable periods	Coverage line always shown; absolute words rejected when coverage < 1; comparisons need ≥ 3 documented days and ≥ 40% coverage per period.
V8–V9	Correct values and direction	Derived fields recomputed; stated direction must match the backend sign; below the minimum change it must read "no clear change".
V10	No causation	Rejects causal and predictive wording ("because", "leads to", "will", ...).
V11–V14	Consistency and safety	Finding kind matches the tool; no URLs or markup; status consistent; charts only from compatible evidence.

On failure the model gets **one** repair turn containing only error codes. If it fails again, invalid findings are dropped (status *partial*) or, if none remain, the owner's fallback text is shown. Unvalidated model text is never displayed. The OpenAI client sits behind an injected `ModelClient` port (Responses API, `store=false`, 30 s per call, 75 s per question, a 40,000-token budget, 20 questions per hour, one in flight) with privacy-safe logging.

4. Claude Code methodology

Role	Responsibility	Constraint
Owner (human)	Product direction, approvals, corrections, external accounts.	Approval is never inferred from silence.
Architect (Claude agent)	One bounded design decision at a time, with options and tradeoffs.	Read-only tools; no code.
Coder (Claude agent)	Implements one approved task with focused tests.	Never certifies its own work.

Verifier (Claude agent)	Runs the suite, black-box and adversarial probes, and Playwright at two viewports; reports PASS or FAIL with evidence.	Read-only on the code; starts from the acceptance criteria, not the Coder's summary.
Integration Lead (main session)	Orchestrates tasks, owns the plan and log, sole writer to `main`, runs an integration Verifier after merges.	Material changes return to the owner.

These build-time Claude agents are distinct from the *runtime* OpenAI agent inside the product. Their definitions are committed in `.claude/agents/`, and `CLAUDE.md` states the workflow every session follows.

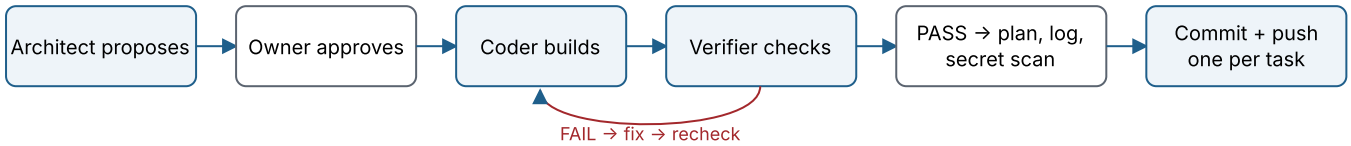


Figure 4. Phase B loop. A task is *Done* only with recorded passing evidence.

Human gates

Phase A ran on 16 Oct: scope, stack, schema, AI sources and security, then UI and test matrix, each approved, amended or corrected by the owner before **GATE-A** authorized implementation. Later material changes (the data-preservation policy, the product refocus, soft delete, the 30-day dataset, the public release) each have a recorded owner decision. Paid API use, dataset approval and publication were never decided by an agent.

Source-of-truth discipline

`PROJECT_PLAN.md` holds task state with *Completed / Not completed / Verification / Evidence* fields; `DEVELOPMENT_LOG.md` records prompts, decisions, defects and observed results; a resume checkpoint let new sessions continue without conversational memory. Unverified claims stay labelled as such.

Parallel workstreams

From LOG-019 the work ran in three isolated checkouts under a written protocol (`docs/architecture/parallel-work-and-metrics-contract.md`):

Workstream	Scope	Rule
Integration Lead	Demo Mode → metrics → dashboard → integration	Only writer to `main` and the planning documents
Analytics worker	Tools, validator, agent, Analyst UI	Built against a **frozen metrics contract** and an in-memory fake until the registry merged
Submission worker	Report draft, audit, release procedure	Documentation only

Exclusive file ownership made merges clean; the cost was that analytics tools were verified against a fake until integration, which is why an integration Verifier ran on the merged result.

5. Prompts and iterations

Appendix B presents ten curated interactions, selected for engineering value rather than volume. Each states the challenge, the original prompt (quoted verbatim from the recorded session where available, and labelled), what the AI produced, the owner's judgment, the iteration and verification that followed, and the takeaway. The full sanitized entries are in `docs/submission/prompts/`. Recurring patterns:

- **Prompts as contracts.** The most effective prompts stated acceptance criteria and prohibitions ("Wait for my explicit approval", "Do not merge B-15 until the isolation fix passes verification") rather than implementation steps.
- **Human correction of plausible AI framing.** The Architect presented AI calorie re-estimation as an acceptable exception to "AI never edits a saved record"; the owner rejected the framing and the design became append-only before any code existed.
- **Agents checking agents.** The main session verified an Architect claim against git history and found it false; Verifiers repeatedly rejected green Coder work.

- **Honest status under time pressure.** Release requests asked for "exact blockers" and "do not represent unfinished work as complete"; the release notes therefore list what was not run.

6. Verification and testing

6.1 Test strategy

Layer	What it covers	Status
Unit	Configuration, canonical keys, URL and SSRF rules, dates, metric registry, analytics schemas, ledger, renderer, charts, validator rules and boundaries.	TESTED
Integration	Constraints, migrations with real multi-process concurrency, data preservation, two-user isolation, soft-delete lifecycle, demo/personal exclusivity, fixture checksum and import, dashboard aggregates.	TESTED
Security	ID-token, session and CSRF probes; loopback-only test login; proxy-header refusal in Demo Mode; static tests for raw connections and string-built SQL; read-only connections deny writes, PRAGMA, ATTACH and VACUUM.	TESTED
Browser	Playwright by the Verifiers at 1440×900 and 390×844: tabs, add, duplicate, edit, delete, restore, history windows, aggregates, charts, Analyst card, keyboard path, overflow, demo entry and reset.	TESTED per task · PENDING committed E2E suite (B-10)
Scripted model	Agent loop, tool dispatch, caps and timeouts, repair and degrade, adversarial final answers, injected record text, error messages, privacy-safe logs, no-key state.	TESTED
Golden (offline)	About 20 questions over the fixture, expected values from an independent reference calculator, pass bar 100%.	PENDING
Live evaluation	Real model behaviour, quality, latency and cost.	NOT RUN (owner approval O-9)

6.2 Recorded results

Checkpoint	Commit	Verifier rounds	Recorded result
B-01 scaffold, B-03 schema	`42c21af`	FAIL → PASS → PASS; FAIL → PASS	75 passed; ruff and mypy --strict clean.
F-01 data-preservation safeguards	`42c21af`	FAIL ×5 → PASS	326 passed in 5/5 runs; 28 earlier bypass repros rejected; unicode-whitespace fuzz 0 bypasses.
B-02 sign-in	`e1f1984`	FAIL → PASS → PASS	380 passed (3 runs); 61 ID-token, 52 session/CSRF and 61 data-preservation probes, including 6 processes × 4 threads.
B-04 data operations	`dc0cfa8`	FAIL → PASS	690 passed; Playwright at both viewports; 12 same-key creates gave 1 × 303 and 11 × 409.
B-14 Demo Mode	`f06e00d`	PASS	853 passed, 1 skipped (×3); personal database bytes unchanged under demo; 403 for 8 proxy headers; 0 network connects without a key.
B-05 metric registry	`fcc64ba`	PASS	1,016 passed; 119,256 randomized and 53,704 fixture checks against an independent calculator, 0 mismatches; 56 forbidden statements refused; 5 injected mutations caught.
B-13 analytics core (worker branch)	`ee976b4`	FAIL → FAIL → PASS	1,350 passed; decoy-user isolation; injection strings; read-only denial with unchanged database hash.
Integration: B-07, B-13, B-15 on B-05	`32d849a`	FAIL → PASS	About 1,740 passed (×2). Real tools on real metrics over the fixture: sleep halves 7.4615 vs 6.5714 h, equal to the reference calculator; untracked domain and non-comparable periods gave the fallback; wrong literal number and "because" rejected. Demo isolation held before and after a reset. Playwright at both viewports.

Never run in this project: live OpenAI calls, the golden set, live Google sign-in, real Cloudflare tunnel headers, the Windows PowerShell start and stop scripts, a screen-reader pass and measured contrast. A Windows clean clone on Python 3.11 was run once by the submission worker (Appendix C); its three findings were fixed but not re-run on Windows.

6.3 Scripted-model tests and the live evaluation plan

Validated with a scripted model	Not validated without a live model
• Tool arguments are schema-checked; a model-supplied `user_id` is rejected. • Turn, call, token and time limits stop the loop. • Every number shown comes from the backend; invented numbers, causal words and undocumented-day claims never reach the page. • One repair turn, then partial or fallback, exactly as specified. • Record text with injection strings never reaches the answer. • Only the signed-in user's data, and in Demo Mode only the demo copy, is read.	• Whether a real model picks the right tools and periods. • How often it follows the placeholder protocol, and so how often answers degrade to partial or fallback. • Readability and usefulness of real answers. • Latency, token use and cost per question. • Compatibility of the request with the live API beyond its tested shape.

Live evaluation plan (requires the owner's approval and a key with a hard budget): run the golden set and the adversarial set against one pinned model with `store=false` on the demo fixture; record per question the status (answered, partial, fallback), validator rejections and repairs, evidence recall against the reference calculator, latency and tokens; report the model name and date; label every result as live; never let a live result replace an offline assertion.

7. Defects found by independent verification

The Coder's own test suite was green before each of these rounds. Each defect was found by an independent Verifier (or by the owner, for design errors), fixed, and rechecked. The full list of more than 30 findings, including non-blocking ones, is in the evidence index §2.

Task, round	Defect	Root cause	Correction
A-03 (design)	AI calorie re-estimation framed as an exception to "AI never edits a saved record".	Plausible framing of a rule violation.	Owner correction: append-only estimates and a summing view, before any code.
B-01 R1	A test login could pass its localhost guard while publicly exposed through the tunnel (High); configuration errors echoed secret values.	Start order assumed the public URL; default error messages.	Tunnel-first start; value-free settings errors; placeholder secrets rejected.
F-01 R1–R5	Five consecutive bypasses of the data-preservation guards: `;COMMIT;` inside a migration, `UPDATE … SET rowid=77`, a `WHEN 0` guard, a double-quoted WHEN clause read by SQLite as a string, standing deletion grants inserted by a migration.	Guards were verified by **matching trigger text**, which an adversary can mimic.	SQLite authorizers, exact guard templates, invariants before COMMIT and at startup. R6 PASS.
B-02 R1	Test login trusted the client `Host` header: a remote request with `Host: localhost` received a session.	"Local" decided from a header, not the peer.	Also require a loopback peer address; five new tests failed on the old code.
B-02 R2	A client disconnect mid-form let the CSRF middleware pass a truncated body as complete.	Body replay did not track completeness.	Partial bodies dropped; R3 PASS on raw sockets.

B-04 R1	Every real-browser add and edit returned 403 while 607 tests passed.	Jinja macros imported without context left the CSRF token empty; tests read the token from a meta tag.	`with context`; new test submits each rendered form's own fields (13/13 fail when reverted).
B-13 R1–R2	"You lost 1.5 kg" accepted on a weight *gain*; direction words with no change evidence; value findings bypassing coverage rules.	Word lists instead of stems; words not linked to evidence.	Signed delta rendering, stem lexicon, direction words must be backed by a sign-bearing placeholder. R3 PASS.
B-15 (worker)	In Demo Mode the Analyst opened the personal database path.	Connection factory ignored the mode.	Mode-aware, fail-closed read-only connection; regression test across a demo reset (Appendix B #9).
Integration R1	Comparison and "Show data" tables overflowed at 390 px.	Wide tables without scroll regions.	Scroll regions with keyboard access; R2 PASS on all 12 aggregate pages.
Windows clone	Demo Mode refused to start on a default Windows clone (fixture checksum mismatch).	Git converted the checksummed JSON to CRLF.	`.gitattributes` keeps the fixture bytes; not yet re-run on Windows.

Pattern. The most valuable findings were not crashes but *false confidence*: guards that looked present, tokens that looked valid, words that looked supported. Independent and adversarial verification exposed each of them.

8. Results and reproducibility

Item	Final state
Foundation: B-01, B-03, F-01, B-02, B-04	TESTED Done with Verifier PASS (`42c21af`, `e1f1984`, `dc0cfa8`).
Demo Mode (B-14), metrics (B-05)	TESTED Done with Verifier PASS (`f06e00d`, `fcc64ba`). Synthetic dataset approval by the owner (O-8): **pending**; the fixture carries `"approved_by_owner": false`.
Dashboard (B-07), analytics (B-13, B-15)	TESTED Integrated in release `32d849a`, integration Verifier PASS in round 2. NOT RUN live OpenAI; golden set pending.
Deferred	B-06 AI intake, B-08/B-09 external agents; B-10 committed E2E suite and B-11 tunnel check not built.
Public release	Single-commit snapshot of the release, published with the owner's approval; private history preserved unchanged; this report, the sanitized records and the curated prompts added as documentation commits; video attached to release `submission-v1`.

Reproduce the demo (no Google account, no API key)

1. Clone the public repository, create a Python 3.12 virtual environment (recommended; 3.11+ allowed) and run `pip install -e ".[dev]"`.
2. Start Demo Mode: `scripts\start.ps1 -Demo` on Windows, or `APP_MODE=demo python -m uvicorn app.main:create_app --factory --host 127.0.0.1 --port 8000` on any platform (README, "Demo Mode").
3. Open `http://localhost:8000` and choose "Explore demo data". The Analyst shows the no-key message; with a reviewer's own key it runs on the synthetic data.
4. Check reproducibility: `python scripts/build_demo_fixture.py --check` regenerates the fixture and compares bytes; `pytest` runs the suite.

Demo video (download): 4 min 42 s, 1280×600 H.264 with the original audio. The browser's top bar was cropped for privacy; nothing else was edited. It shows Demo Mode without an OpenAI key and is not evidence of live AI.

9. Critical reflection and risks

What worked

- **Independent verification paid for itself.** F-01 needed six rounds; each FAIL exposed a bypass that a self-reviewed green suite would have shipped.
- **Recorded human gates** caught a design misframing before any code and kept scope changes explicit.
- **Moving trust from the model to the backend** (placeholders instead of numbers, fixed tools instead of SQL) made AI correctness testable with ordinary deterministic tests.
- **A frozen interface contract** let analytics proceed in parallel with the metric registry.

What I would do differently

- Test forms through their own rendered fields from the start, and verify guards by behaviour, not by text.
- Run the suite on the reviewer's platform early: the Windows line-ending defect was invisible on Linux.
- Budget the golden set and a small, approved live run before the release window, so the AI claims rest on measured model behaviour, not only on the guarantees around it.
- Commit the Verifiers' browser journeys as an E2E suite instead of keeping them as per-task probes.

Tradeoffs accepted

Choice	Cost
One agent + deterministic validator (no RAG, no multi-agent)	Questions limited to what six tools express; single-turn only.
Heuristic thresholds (V5–V9)	Not significance tests; may refuse borderline real effects.
English lexical rules	Best effort; idioms can slip through, although rendered numbers always carry the true sign.
Server-rendered UI without JavaScript	Simple and CSP-friendly, less interactive.
Parallel workstreams	Integration risk concentrated at the end.

Remaining risks and limitations

- Real-model behaviour, answer quality and cost are unmeasured (§6.3).
- "Steps on aerobic days" is not expressible (no tool filter), and a comparison with zero events in one period falls back; both are open design questions.
- Single-user local app; a copied session cookie stays valid until its 8-hour expiry; rate limits are per process.
- Windows fixes and PowerShell scripts not re-run on Windows; no screen-reader or contrast audit.
- The synthetic dataset is not yet owner-approved; some invented organization names may resemble real ones.

10. Conclusion

The project delivers a small, working, reproducible product whose AI component is constrained so that its correctness can be checked by ordinary deterministic tests: the model chooses tools and words, the backend owns every number and every claim. The development process shows the same principle applied to the AI tools that built it: Claude Code agents proposed, implemented and verified, but approvals came from the owner, and each task was closed only on independent evidence. The main open item is equally clear: the live behaviour of the runtime model has not been measured, and the report says so rather than implying otherwise.

Appendix A. Decision register

ID	Decision	Alternatives considered	Approval
A-01	MVP scope: full core with bounded agents (option B)	Full specification; core first	Owner, 16 Oct (amended)
A-02 / ADR-001	FastAPI + HTMX + SQLite, hand-written SQL, Pydantic	TypeScript Fastify + React; Next.js + Prisma	Owner, 16 Oct
A-03	DB-enforced canonical keys; append-only calorie estimates	Plain UNIQUE + app checks; expression indexes	Owner, 16 Oct (E2 corrected)
A-04 / ADR-002, ADR-003	Web search for jobs and trends; quick tunnel; security defaults S1–S12	Jobs/news APIs; named tunnel on own domain	Owner, 16 Oct; agents deferred by ADR-005
A-05	English UI, top tabs / mobile bottom bar, light theme; test matrix and build order	Hebrew UI; sidebar; dark theme	Owner, 16 Oct (GATE-A)
ADR-004	Data preservation: RESTRICT FKs, grant-guarded deletes, migration invariants	Cascades plus convention	Owner directive, 17 Oct
ADR-005	Personal analytics: one function-calling agent, read-only tools, Evidence Validation, Demo Mode	Embeddings/RAG; multi-agent frameworks; LLM-generated SQL; committed demo database	Scope 17 Oct; design 18 Oct (O-0…O-12, O-9 live calls withheld)
LOG-019	Defer AI intake; 30-day fixture; parallel workstreams	Keep intake in MVP; 90-day fixture	Owner, 18 Oct
O-12	Publish a fresh single-commit public repository; keep private history unchanged	Rewrite and publish the private history	Owner, 18 Oct

Sources: `DEVELOPMENT_LOG.md` (LOG-002…LOG-023), `PROJECT_PLAN.md`, `docs/submission/evidence-index.md`, `docs/architecture/`, `docs/submission/prompts/`. Version 2.0 replaces version 1.0 of this report (kept in the repository history).

Appendix B. Claude Code prompts, iterations and engineering decisions

Ten interactions selected for engineering value. Owner prompts are quoted verbatim from the recorded session (elisions marked [...]); each block below is condensed from the full entry in `docs/submission/prompts/`, which also gives the AI's actual response, the iteration history and the sources. The analytics and submission workers ran outside the recorded project, so their prompts are not available; their work is evidenced only by committed notes.

01. Phase A kickoff and the first scope decision (A-01)

PHASE A (DESIGN ONLY) / A-01 MVP SCOPE · DATE: 2026-10-16, 10:47–10:56 UTC · VERBATIM RECORDED PROMPT

Challenge. Start an AI-assisted project without letting the assistant jump straight to code. The first design decision (MVP scope) had to be proposed with alternatives, and nothing could proceed without an explicit human approval.

> *Read CLAUDE.md, PRODUCT_SPEC.md and PROJECT_PLAN.md. Start Phase A with the Architect agent only. Propose the first architecture decision with alternatives and tradeoffs. Wait for my explicit approval before proceeding. Do not write application code.*

Human judgment. The owner approved B but changed its behaviour:

> *approve B. i want it to be in a way that the Jobs and trends are recommended each time the user clicked the button "Run now". its possible to do it more than once a day, but it can be up to 3 times a day.*

Takeaway. Having the assistant stop after one bounded decision gave the owner room to amend it. Asking the follow-up question then surfaced a requirement conflict (the scheduled run against the manual cap) that would otherwise have been settled silently by default. Full entry.

02. Calorie re-estimation: correcting an "exception" into an append-only design

PHASE A / A-03 SCHEMA (EDGE CASE E2) · DATE: 2026-10-16, 10:58–11:15 UTC · VERBATIM EXCERPT OF A RECORDED PROMPT

Challenge. The project rule was that the runtime AI never overwrites saved records. The owner also wanted meal calories to be re-estimated whenever a meal changes. The Architect resolved the tension the wrong way, and the error had to be caught before any schema was built.

> *approve A, but E2 needed to be treated more deeply - i want the LLM to calculate the calorie each time a meal is added, edited or deleted on fly, which means that the LLM gets the prompt, extract items, and if it found a meal within the prompt, it calculate the calories and it will be added to the relevant field. [...]*

Human judgment. The owner rejected the "exception" framing. A rule that admits a convenient exception stops being a rule. The owner restated the requirement as additive: every change creates a new record, and the daily total is derived. The owner then chose "Estimates table" (11:15 UTC). Main Claude logged the misframing as its own error: "I also logged that the "exception" wording was my mistake, which you caught."

Takeaway. Explaining a requirement as an "exception" was a sign that the model had not yet found a design that satisfied the rule. An append-only data model satisfied both constraints at once, and it also produced an audit trail. Full entry.

03. Data-integrity directive: ADR-004 and six Verifier rounds

PHASE B / F-01 FOUNDATIONAL SAFEGUARDS (INSERTED BEFORE B-02) · DATE: 2026-10-17, 18:45–19:54 UTC · VERBATIM EXCERPT OF A RECORDED PROMPT

Challenge. A read-only schema review had found residual risks (L1–L4), including one where `INSERT OR REPLACE` could silently delete a user's data. The owner made data preservation a project-wide requirement: no automatic deletion during sign-in, updates or migrations, and safe migrations under real concurrency.

> *1. Data Integrity and Deletion Policy — Highest Priority Data preservation must be a fundamental architectural requirement throughout the entire project. * No automatic or unintended deletion of user data is allowed during authentication, updates, synchronization, migrations, or error handling. [...] * Prevent destructive SQL operations, including unsafe INSERT OR REPLACE patterns. [...] Do not implement new deletion features [...]*

Human judgment. The owner set the policy and the execution order and approved routine decisions within scope. Main Claude adopted some defaults without escalating them and recorded each one (LOG-011):

- `RESTRICT` plus grant-guarded deletes;
- the soft-delete design deferred to B-04 for owner approval;
- frontend URL validation deferred until forms exist.

Takeaway. Each fix created new attack surface, and only repeated adversarial re-verification found it. Checking what SQLite actually does (authorizers, behaviour) proved sturdier than comparing SQL text. Full entry.

04. Google sign-in: identity preservation and a Host-header trust bug

PHASE B / B-02 GOOGLE OAUTH, SESSION AND CSRF · DATE: 2026-10-17, 19:59–20:31 UTC · VERBATIM EXCERPT OF A RECORDED PROMPT

Challenge. Sign-in is where data-loss bugs often hide: a repeated or concurrent first login can re-create a user row and orphan that user's records. The owner also wanted the OAuth setup fully configurable, with no personal credentials in the code.

> *[...] Continue with B-02 as planned, following our established Architect → Coder → Independent Verifier workflow. Make sure the Google OAuth implementation is fully configurable through environment variables, without hardcoded personal credentials or email addresses. The Verifier must explicitly test that repeated Google sign-ins preserve the existing user identity and all associated data, including relevant concurrent [...]*

Human judgment. The owner set the acceptance bar: identity had to survive repeated and concurrent sign-ins. Main Claude took three defaults without escalating them and recorded each one (LOG-013):

- an absolute rather than sliding 8-hour session;
- the CSRF token also accepted as a form field, so sign-out works without JavaScript;
- a separate `test-auth:` identity, so a test user can never act as a real Google user.

Takeaway. A "localhost only" guard based on request data the client controls offers no protection. The Verifier also kept to the requirement as stated, even though the preconditions made the bug low-impact. Full entry.

05. Product direction change: Personal AI Analytics (ADR-005)

ARCHITECTURE RE-SCOPE, A-06 / ADR-005 (PLANNING ONLY) · DATE: 2026-10-17 21:22 UTC; CHECKPOINT PUSHED 23:55 UTC · VERBATIM EXCERPT OF A RECORDED PROMPT

Challenge. The owner changed the product's purpose: from external job and industry-trend recommendations to analysis of the user's own records. The new core feature was a runtime agent that answers free-form questions about personal data. That agent must not run arbitrary SQL, make up numbers or claim causes, and the change had to be designed and documented without touching code.

> *IMPORTANT: This session is for architecture and planning only. Do not invoke the Coder or Verifier, implement features, modify application code, install dependencies, or begin B-04. [...] This is NOT a decision driven by token limitations, development cost, or time constraints. [...] Do not allow the LLM to execute arbitrary SQL. Instead, design bounded, read-only, parameterized retrieval and analytical tools. The model may [...]*

Human judgment. The scope was decided by the owner and not re-asked. Main Claude did not accept the Architect's draft without checking it. One claim was that git commit metadata contained the owner's email. Main Claude checked the history, found that every commit is authored by the Claude tool identity, and corrected ADR-005. As reported to the owner: "One correction: the Architect said the commit history contains your email. I checked, and every commit is authored as Claude, so I fixed that in ADR-005." The recommendation for a fresh public repository stayed, with its real reasons recorded instead: a company name in an early commit, and private-session details in the log.

Takeaway. An Architect agent's review can contain confident factual errors about the repository. The orchestrator checked a claim that would have shaped the release strategy against git before passing it on. Full entry.

06. ADR-005 approvals: soft delete, offline-only evaluation, owner-held dataset approval

ADR-005 OPEN DECISIONS O-0 TO O-12; PHASE B RESUMES AT B-04 · DATE: 2026-10-18 03:03 UTC · VERBATIM EXCERPT OF A RECORDED PROMPT

Challenge. The Architect had proposed a default for each of 13 open decisions. Some of them concerned cost, data ownership and publication, which are decisions an autonomous implementation session should not make. The owner had to approve the design, while keeping the decisions with real-world consequences under human control.

> *I approve ADR-005 (O-0), subject to the decisions below. I also approve the following: * O-1: Option A — recoverable soft deletion using `deleted_at`, a Recently Deleted view, explicit restoration, and separately confirmed permanent deletion. Preserve all ADR-004 data-integrity safeguards. [...] * O-8: The Coder may prepare the synthetic dataset according to ADR-005. I will review and approve the final dataset before [...]*

Human judgment.

- **O-1:** the owner accepted the recommended soft-delete design and restated that ADR-004 stays binding.
- **O-9:** the owner did not take the Architect's default of a small live run. Only offline, mocked evaluation was authorized, and any paid call needs prior approval. This kept spending and the use of the owner's credentials under owner control.
- **O-8:** the Coder could draft the synthetic data, but approval of the final dataset stayed with the owner.

Takeaway. Delegating implementation did not mean delegating every decision. Paid external calls and approval of published data stayed explicit owner gates, and the records keep work that ran apart from claims that were never tested. Full entry.

07. 607 passing tests and every browser form broken: the B-04 CSRF blind spot

PHASE B / B-04 DATA OPERATIONS (MANUAL CREATE, EDIT, SOFT DELETE, RESTORE, HISTORY) · DATE: 2026-10-18, 03:05–04:33 UTC · DOCUMENTED DECISION WITHOUT AN AVAILABLE ORIGINAL PROMPT

Challenge. The Coder reported B-04 as complete, with 607 tests passing, ruff and mypy clean, and a curl walkthrough on a real server. The question was whether a green suite meant a working product.

> *The Independent Verifier must evaluate complete user journeys, not just individual functions.*

Human judgment. Main Claude treated the miss as a flaw in how the tests were designed, not only as a template bug, and accepted the Verifier's suggested fix: a regression test that submits each rendered form's own fields. To unblock browser testing, Main Claude added a wrapper script for the preinstalled Chromium. The change was to the environment only, not the repository. The Coder's assumption about task dates (past dates allowed, up to today+14) was accepted by the orchestrator and recorded.

Takeaway. Tests that use the same shortcut as the implementation, here the meta-tag token, can stay green while the user path is broken. Black-box probing of rendered output found what the white-box suite could not. Full entry.

08. Parallel workstreams: a frozen metrics contract and a single integrator

PLANNING AMENDMENT DURING B-14; B-05 CONTRACT; B-13/B-15 ANALYTICS WORKSTREAM · DATE: 2026-10-18 04:47 UTC; PROTOCOL PUSHED 04:53 UTC · VERBATIM EXCERPT OF A RECORDED PROMPT

Challenge. The owner wanted a second, independent Claude Code thread to build the analytics agent (B-13/B-15) while the main thread finished Demo Mode, the metrics registry (B-05) and the dashboard. Two AI workers editing one repository can easily overwrite each other's work or merge unverified code.

> *I intend to start a second independent Claude Code thread for B-13/B-15. You are the integration lead and the only worker authorized to integrate verified changes into main. The analytics worker must use an isolated worktree or checkout and a separate feature branch. Coordinate the B-05 metric registry contract with the analytics worker. Avoid overlapping edits. Do not let both workers modify shared application files or [...]*

Human judgment. The owner set the boundaries: one integrator, isolated branches, a verifier per task plus an integration verifier. The owner also corrected the plan with facts from outside this session. The analytics branch was actually `feature/analytics-agent`, not the documented `feature/analytics-b13`. The owner warned: "Do not treat B-13 as fully integrated or production-ready merely because its branch was pushed."

Takeaway. Parallel AI work stayed safe because the coordination rules were written down in the repository (ownership, a frozen interface, one merger) and did not depend on conversation memory. The contract let the analytics worker build against a fake backend and integrate later. Full entry.

09. Demo-mode isolation defect in the Analyst: fail closed, prove it across a reset

B-15 INTEGRATION (ANALYTICS BRANCH INTO `MAIN`) · DATE: 2026-10-18, 06:18–06:55 UTC · VERBATIM EXCERPT OF A RECORDED PROMPT

Challenge. Demo Mode (B-14) promises that a reviewer's session never opens or changes a personal database. The new Analyst route came from a separate workstream and broke that promise: in demo mode it opened `DATABASE_PATH`. The defect had to be fixed and proven before the branch reached `main`.

> *The Analytics worker discovered a B-15 database isolation defect. Issue: The new `POST /analyst/ask` route was opening the personal database (`DATABASE_PATH`) instead of the active synthetic Demo Mode working copy when `APP_MODE=demo`. [...] * The Analyst must obtain its database connection through the application's mode-aware database context. [...] * Add regression tests proving that Analyst queries in Demo Mode never open or [...]*

Human judgment. The owner held the merge gate and named the required evidence: regression tests, including a reset. The record keeps the verification status of each layer apart. The worker's notes record its fix as "Tested by the Coder and the orchestrator's gate run; **not re-verified by the Verifier**". The owner later reported that both analytics tasks had passed their independent Verifier, and said that "real B-05 integration, golden tests and Playwright checks are still required." The integration lead then had the integration Verifier probe isolation independently instead of relying on either report.

Takeaway. When one mode must never touch another mode's data, the safe default is to refuse to run, not to fall back. The regression test proves the guarantee across a reset, and that is where a naive cached path would have failed. Full entry.

10. Release readiness under time pressure: a FAIL, a fix and an explicit "not run" list

INTEGRATION OF B-07 + B-13/B-15 WITH B-05; RELEASE TO `MAIN` · DATE: 2026-10-18, 06:07–06:55 UTC · VERBATIM EXCERPT OF A RECORDED PROMPT

Challenge. The owner needed an integrated, demonstrable MVP within a short, fixed window and asked for a yes or no on release readiness. Under that pressure, the obvious failure mode is to call the build ready and leave out what was never tested.

> *Verify the integrated path: Demo login → synthetic data → metrics/dashboard → Analyst question → read-only tools → evidence validation → response/chart. Use Playwright at desktop and mobile viewports and run the appropriate automated regression/security tests. [...] If an essential component is not ready, communicate its exact status and arrange an evidence-based fallback. Never mark an unverified capability as complete. Do not [...]*

Human judgment. The owner had set the bar that safety properties were not negotiable under time pressure, and that unverified claims were not allowed. Two of the Verifier's findings were left as open owner and Architect decisions rather than patched under pressure:

- adding a filter to the T5 tool, which would be a data-contract change;
- validator rule D-6 rejecting a comparison where one period has 0 events.

Takeaway. A readiness decision is only useful if it comes with its limits. Here that meant naming the latest safe commit, the check still in progress, and exactly which features must not be shown as working. Full entry.

Appendix C. Additional evidence

C.1 Windows clean-clone run (submission worker, Python 3.11.9)

ID	Finding	Cause and status
W-1	As cloned: 95 failed, 753 passed, 2 skipped, 4 errors; Demo Mode refused to start ("Demo fixture checksum mismatch").	Git converted the checksummed fixture to CRLF. Fixed in `32d849a` with `.gitattributes`; not re-run on Windows.
W-2	One migration test failed under the default Windows locale.	`write_text()` without an encoding. Fixed in the test; not re-run on Windows.
W-3	One cleanup test failed: creating a symlink needs a Windows privilege.	Test environment limitation; the case now skips when symlinks are unavailable.

With an LF checkout the same run gave 850 passed, 2 failed, 2 skipped; with `PYTHONUTF8=1`, 851 passed and 1 failed. ruff and mypy --strict were clean.

C.2 Synthetic dataset

The fixture `demo/synthetic_v1.json` is generated by a seeded script (`scripts/build_demo_fixture.py --check`) and locked by SHA-256. Dates are stored as day offsets and shifted at import so the data always ends today. It describes one fictional person.

Domain	Rows	Domain	Rows	Domain	Rows	Domain	Rows
Tasks	89	Meals	70	Sleep	27	Steps	27
Workouts	15	Articles	12	Series episodes	10	Job applications	9
Weight	8	Podcasts	8	Movies	2	≈10% of days missing per domain	

Patterns that must be found (asserted by `tests/support/reference.py`)

- Friend meetings twice weekly in weeks 1–2, none in weeks 3–4
- Interview tasks per week strictly increasing
- Workouts and family meetings about 3–4 per week
- Higher steps on workout days (association)
- A "no clear change" control; sleep halves differ above the threshold

Cases that must yield partial or insufficient

- Untracked domain (mood, heart rate)
- Movie trend (only 2 movies)
- A specific undocumented date
- Weight trend over the last week
- A comparison where one period lacks coverage

C.3 Security and privacy controls

Threat	Control
Prompt injection via stored text	Read-only, user-scoped tools; stored text is never addressable by a placeholder; the model cannot place digits, URLs or markup in an answer.
Cross-user leakage	Every query binds the session user; tool models forbid extra fields; decoy-user isolation tests.
SQL misuse	No model-generated SQL; static tests; read-only authorizer; 2-second budget.
Cost abuse	Token, call, turn and hourly caps; one question in flight; reviewers use their own key with a budget limit.
Demo exposure	Localhost-only Demo Mode; 403 for proxy and tunnel headers; tunnel URLs refused.
Secret and PII leakage	Value-free configuration errors; `.gitignore` for env files, databases, documents, media and traces; secret and personal-data scan before every push.

A tree-and-history audit of the private repository found no live credential, personal email, real record, database or screenshot. Two history-level items (a company name in the first commit, private session links in commit messages) led to publishing a fresh single-commit snapshot instead of the private history. Redactions applied to the published development records are listed in `docs/submission/redactions.md`.